TradeFlow MVP

Complete AI-Ready Project Specification & Build Roadmap

A real-time trading simulator built with React, Node.js, TypeScript, NestJS, PostgreSQL, Redis, BullMQ and WebSockets.

Purpose: This document is designed to be given to an AI coding assistant, used as a project specification, or followed manually to build TradeFlow step by step.

1. Executive Summary

TradeFlow is a simulated real-time trading platform. Users can manage a virtual cash balance, view simulated market prices, place buy and sell orders, receive real-time order updates, and track a virtual investment portfolio.

The project deliberately focuses on realistic asynchronous business workflows rather than being another basic CRUD application. An order can move through states such as NEW, PROCESSING, PARTIALLY_FILLED, FILLED, REJECTED, and CANCELLED.

> **Important:** TradeFlow does not connect to a real stock exchange and does not process real money. All instruments, prices, balances, and executions are simulated.

2. Why Build This Project?

• Demonstrate full-stack engineering beyond simple CRUD applications.

• Show React and Node.js/TypeScript skills in addition to existing enterprise development experience.

• Demonstrate real-time communication using WebSockets.

• Model complex asynchronous workflows and state transitions.

• Demonstrate database consistency, partial executions, idempotency, background jobs, authentication, and authorization.

• Create a polished public project that recruiters can run, explore, and inspect on GitHub.

• Provide a strong discussion topic for system design and backend interviews.

3. Core Product Story

A user places an order. The platform validates the request and creates the order. The order is then sent asynchronously to an Exchange Simulator. The simulator can partially fill, fully fill, reject, or eventually cancel the order. Every important state change is persisted and pushed to the user in real time. Successful executions update the user's portfolio.

```
User places order
      ↓
Order validation
      ↓
Order created as NEW
      ↓
Background job / Exchange Simulator
      ↓
PARTIALLY_FILLED / FILLED / REJECTED
      ↓
Database updated + execution recorded
      ↓
WebSocket event emitted
      ↓
React UI updates without refresh
      ↓
Portfolio and positions updated
```

4. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	UI and type-safe frontend development
Build Tool	Vite	Fast local development and production builds
Routing	React Router	Client-side routing
Server State	TanStack Query	API data fetching, caching and synchronization
Client State	Zustand	Lightweight UI/client state
Real Time	Socket.IO Client	Receive price and order updates
UI	Tailwind CSS + shadcn/ui	Modern responsive interface and reusable components
Charts	Recharts	Portfolio and market visualizations
Backend	Node.js + NestJS + TypeScript	Structured modular backend
Database	PostgreSQL	Relational persistence
ORM	Prisma	Type-safe database access and migrations
Queue	BullMQ	Asynchronous order processing
Cache/Queue	Redis	Queue backing and future caching
Auth	JWT + bcrypt or Argon2	Authentication and password protection
API Docs	Swagger / OpenAPI	Interactive API documentation
Testing	Jest	Unit and integration tests
Containers	Docker + Docker Compose	Consistent local environment
CI/CD	GitHub Actions	Automated testing/build pipeline

Technology Decision

Use a modular monolith for the MVP. Do not begin with many microservices. The project should first prove correct business behavior, clean module boundaries, and reliable asynchronous processing. Services can be extracted later when there is a justified reason.

5. User Types

Trader

• Register and log in.

• View simulated instruments and live price updates.

• Place buy and sell orders.

• View open, completed, rejected, and cancelled orders.

• Cancel eligible orders.

• Receive real-time order notifications.

• View virtual cash, holdings, portfolio value, and profit/loss.

Administrator

• View users and platform activity.

• View all orders and execution activity.

• Manage simulated instruments.

• Enable or disable instruments.

• Monitor failed or rejected processing.

MVP Scope Decision

Implement the Trader experience first. An Admin account and basic administrative capabilities can be seeded initially. Avoid spending significant MVP time on a complex admin dashboard.

6. Functional Requirements

FR-01: User Registration — The system shall allow users to register using name, email, and password. Email uniqueness shall be validated and passwords shall never be stored in plain text.

FR-02: Authentication — Users shall be able to log in and obtain JWT-based authenticated access. Protected endpoints shall require valid authentication.

FR-03: Market Instruments — Users shall be able to view simulated instruments containing symbol, company name, current price, and status.

FR-04: Live Market Prices — The system shall periodically generate simulated price changes and broadcast updates through WebSockets.

FR-05: Place Orders — Traders shall be able to submit BUY and SELL orders using symbol, side, quantity, order type, and price where required.

FR-06: Order Validation — The system shall validate instrument existence, positive quantity, supported order type, sufficient virtual cash for buys, and sufficient holdings for sells.

FR-07: View Orders — Traders shall view open, completed, rejected, and cancelled orders with execution progress.

FR-08: Cancel Orders — Traders shall be able to cancel eligible orders that are not already fully executed.

FR-09: Asynchronous Execution — Orders shall be processed asynchronously by an Exchange Simulator using a background queue.

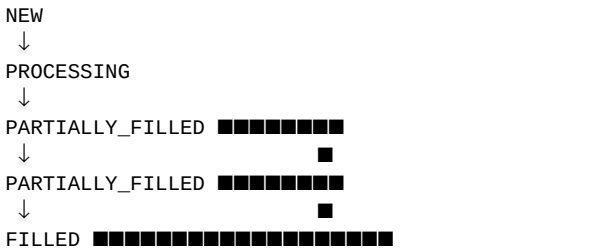
FR-10: Partial Fills — The Exchange Simulator shall support partial executions before an order is completely filled.

FR-11: Portfolio View — Traders shall view available cash, positions, average cost, current market value, and unrealized profit/loss.

FR-12: Portfolio Updates — Successful executions shall update positions and balances safely.

FR-13: Real-Time Notifications — The UI shall receive updates for order creation, partial fills, fills, rejection, cancellation, and market price changes.

7. Order Lifecycle




```
Alternative terminal states:
NEW / PROCESSING → REJECTED
NEW / PROCESSING / PARTIALLY_FILLED → CANCELLED
```

Example: BUY 10 TFLX @ $100

```
Initial order:       10 shares
Execution 1:          3 shares
Remaining:            7 shares

Execution 2:          4 shares
Total filled:         7 shares
Remaining:            3 shares

Execution 3:          3 shares
Total filled:        10 shares
```

```
Final status:         FILLED
```

8. Non-Functional Requirements

NFR-01: Performance — API responses should be efficient under normal load and real-time events should be delivered with low latency. Record actual benchmark results later instead of inventing numbers.

NFR-02: Security — Use password hashing, JWT authentication, role-based authorization, input validation, authentication rate limiting, secure environment variables, and no committed secrets.

NFR-03: Reliability — Failed background jobs should be logged and retryable. Important state changes must be persisted. The system must prevent duplicate execution processing.

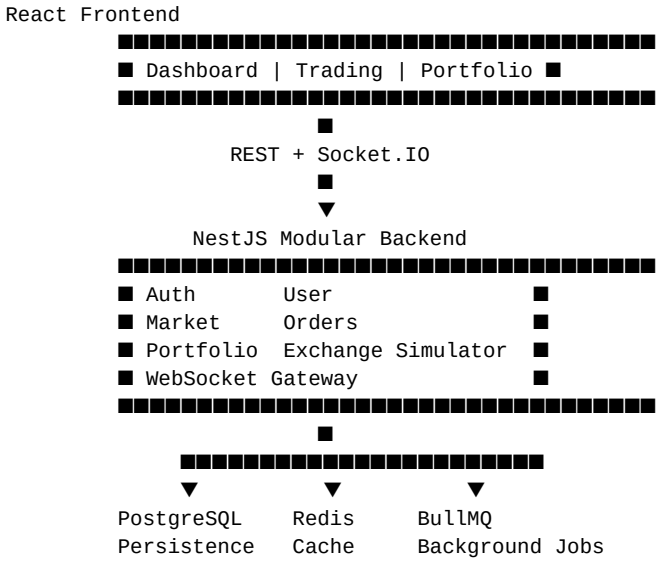
NFR-04: Idempotency — Processing the same execution event more than once must not update a portfolio twice.

NFR-05: Scalability — Module boundaries should allow future extraction of Order, Exchange, Market Data, and Portfolio services.

NFR-06: Maintainability — Use TypeScript, modular architecture, DTO validation, clear separation of concerns, consistent error handling, tests, and API documentation.

NFR-07: Observability — Add structured logging and request/correlation IDs. Prometheus, Grafana, and distributed tracing are Version 2 enhancements.

9. Project Architecture



Recommended backend modules:

• AuthModule

• UsersModule

• MarketModule

• OrdersModule

• ExchangeModule

• PortfolioModule

• WebSocketModule

• CommonModule for shared guards, errors, logging, and utilities

10. Database Design

```
users
- id, name, email, password_hash, role, created_at

instruments
- id, symbol, name, current_price, status, updated_at

orders
- id, user_id, instrument_id, side, order_type
- quantity, filled_quantity, price, status, created_at

executions
- id, order_id, quantity, execution_price
- execution_time, execution_reference (unique)

portfolios
- id, user_id, available_cash

positions
- id, portfolio_id, instrument_id
- quantity, average_price, updated_at

order_events
- id, order_id, event_type, payload, created_at
```

The unique execution_reference is useful for idempotency. If an execution event is delivered twice, the system can detect that it has already been processed.

11. API Specification for MVP

Authentication

```
POST /auth/register
POST /auth/login
GET  /auth/me
```

Market

```
GET /instruments
GET /instruments/:symbol
```

Orders

```
POST /orders
GET  /orders
GET  /orders/:id
POST /orders/:id/cancel
```

Portfolio

```
GET /portfolio
GET /portfolio/positions
GET /portfolio/transactions
```

WebSocket Events

```
market.price.updated
order.created
order.updated
order.partially_filled
order.filled
order.rejected
order.cancelled
portfolio.updated
```

12. Recommended Repository Structure

```
tradeflow/
■■■ apps/
■   ■■■ web/                # React + TypeScript + Vite
■   ■■■ api/                # NestJS API
■■■ packages/
■   ■■■ shared-types/       # Shared TypeScript types/contracts
■■■ docker/
■■■ docs/
■   ■■■ architecture.md
■   ■■■ api.md
■   ■■■ decisions.md
■■■ docker-compose.yml
■■■ package.json
■■■ README.md
■■■ .github/
    ■■■ workflows/
        ■■■ ci.yml
```

13. Step-by-Step MVP Build Plan

Phase 1 — Foundation

• Create a TypeScript monorepo using pnpm or npm workspaces.

• Create React/Vite application.

• Create NestJS API application.

• Add ESLint and Prettier.

• Create Docker Compose for PostgreSQL and Redis.

• Confirm frontend, backend, PostgreSQL, and Redis run together.

Phase 2 — Database and Authentication

• Create Prisma schema and migrations.

• Implement User, Instrument, Portfolio, and Order entities/models.

• Build registration and login.

• Add JWT authentication and protected routes.

• Create login and registration pages.

Phase 3 — Market Dashboard

• Seed instruments such as TFLX, NOVA, APEX, VERT, and QUANT.

• Create an instrument listing API.

• Build the market dashboard.

• Create a simulated price generator.

• Broadcast price changes every few seconds using Socket.IO.

Phase 4 — Order Entry

• Build BUY/SELL order form.

• Support MARKET and LIMIT order types.

• Validate balances, holdings, quantity, and instruments.

• Persist submitted orders as NEW.

• Build order list and order details screens.

Phase 5 — Exchange Simulator

• Push new orders into a BullMQ queue.

• Create an Exchange Worker.

• Simulate filled, partially filled, and rejected outcomes.

• Generate multiple execution events for partial fills.

• Persist executions and order events.

Phase 6 — Real-Time Experience

• Create a NestJS WebSocket gateway.

• Authenticate socket connections.

• Emit user-specific order updates.

• Update React UI immediately without polling or page refresh.

Phase 7 — Portfolio

• Update positions after executions.

- Calculate average buy price.
- Update virtual cash.
- Calculate market value and unrealized P/L.
- Build a portfolio dashboard.

Phase 8 — Quality and Deployment

- Write unit tests for critical order and portfolio logic.
- Test duplicate execution/idempotency.
- Add Swagger/OpenAPI documentation.
- Containerize the application.
- Add GitHub Actions CI.
- Deploy frontend and backend.
- Record a short demo video and add screenshots to README.

14. Critical Test Scenarios

• A user cannot buy shares without sufficient virtual cash.

• A user cannot sell more shares than they own.

• An order can move through multiple partial fills.

• A fully filled order cannot be cancelled.

• A cancelled order cannot receive new execution processing.

• The same execution event processed twice does not change the portfolio twice.

• A user cannot access another user's orders or portfolio.

• WebSocket clients receive only authorized user-specific events where applicable.

15. What NOT to Build in the First MVP

• Kafka

• Kubernetes

• Many microservices

• Real FIX protocol connectivity

• Real stock exchange integration

• Real money or payment processing

• Complex matching engine

• AI features

• A large admin platform

16. Version 2 Roadmap

• Order amendment / replace workflow.

• More realistic limit order and market matching rules.

• Kafka or another event-streaming platform.

• Separate Market Data, Order, Exchange, and Portfolio services.

• Prometheus and Grafana.

• Audit dashboard.

• Advanced risk checks.

• Event sourcing for selected workflows.

• Load testing and published benchmark results.

17. Recruiter Story

30-second version:

> “I built TradeFlow as a real-time trading simulation platform because I wanted to demonstrate more than basic CRUD development. The project models an asynchronous order lifecycle where an order can be

partially filled, fully filled, rejected, or cancelled. I built it with React and TypeScript on the frontend and Node.js, NestJS, PostgreSQL, Redis, BullMQ, and WebSockets on the backend. I focused especially on real-time updates, data consistency, idempotent execution processing, and clean module boundaries."

Long interview version:

"I wanted to build a public project that demonstrates how I approach complex business workflows without exposing confidential code from my professional work. Instead of creating a standard CRUD application, I designed TradeFlow around an asynchronous order lifecycle. A user places an order, the system validates and persists it, then sends it to an exchange simulator through a background job. The simulator can produce partial fills, full fills, or rejection events. Each event is persisted, the portfolio is updated safely, and the user receives the update in real time through WebSockets."

"One of my design priorities was reliability. For example, execution processing is designed to be idempotent so that a duplicated event cannot update the portfolio twice. I also deliberately started with a modular monolith instead of over-engineering microservices. My goal was to establish correct business behavior and clean boundaries first, while keeping the architecture ready for future extraction into independent services if scale justifies it."

"The project demonstrates end-to-end ownership: frontend architecture, backend APIs, database design, asynchronous processing, real-time communication, testing, containerization, CI/CD, and deployment."

18. AI Build Instructions

When using an AI coding tool, provide this PDF together with the following instruction:

"Build TradeFlow according to this specification. Work in small phases and do not implement the entire system at once. Before writing code for each phase, inspect the existing codebase and propose the minimal files and changes required. Preserve TypeScript type safety, clean architecture, and existing functionality. After each phase, provide setup instructions, tests to run, and a concise summary of what was completed. Do not add technologies or features outside the MVP unless explicitly requested."

Recommended AI workflow:

- Start with Phase 1 only.
- Commit or checkpoint after every completed phase.
- Run the application and tests before moving to the next phase.
- Ask the AI to explain architectural decisions briefly.
- Do not accept large amounts of generated code without running and reviewing it.
- Keep an architecture decision log in docs/decisions.md.
- Use the functional and non-functional requirements as acceptance criteria.

19. Definition of Done for the MVP

- A user can register and log in.
- A user receives virtual starting cash.
- Market instruments are visible.
- Prices change in real time.

- A user can place BUY and SELL orders.
- Orders are processed asynchronously.
- At least some orders demonstrate partial fills.
- The UI updates through WebSockets.
- Successful executions update the portfolio.
- Duplicate execution protection is implemented and tested.
- Critical business logic has automated tests.
- The project has API documentation.
- The application can run locally using documented setup steps.
- A public deployment, screenshots, and demo video are available.

20. Final Portfolio Positioning

TradeFlow should position you as a full-stack engineer who can build systems with complex state transitions and asynchronous workflows—not simply someone who can create forms, tables, and CRUD endpoints.

> Your public story is: **“I build end-to-end systems and think about real-world behavior: asynchronous processing, real-time communication, data consistency, idempotency, reliability, and how an architecture can evolve without unnecessary over-engineering.”**

End of TradeFlow MVP Project Specification